

Continual Safety Alignment in Vision-Language Models via Multimodal Gradient-Based Sample Selection

Extending Continual Safety Alignment (Bach et al., 2026) to Multimodal Architectures (<https://arxiv.org/abs/2604.17215v1>)

Researcher: Aryaman Singh Dev | Advisor: Prof. Thao Minh Le | Penn State University

The Problem: Alignment Drift in Continual Fine-Tuning

1. Confirmed in Text LLMs (Bach et al., 2026)

- Benign fine-tuning unintentionally erodes safety guardrails as a structural property.
- High-gradient samples drive elastic reversion toward unaligned pretraining distributions.
- Standard remedies like gradient clipping fail (ASR remains ~31% at clip=0.5), because clipping dampens step size without altering alignment-reversing directions.

2. Our Research Question for VLMs

- Does the same mechanism govern Vision-Language Models?
- Bach et al. explicitly leave this open (Limitations): 'Extending gradient-based selection to VLMs requires further investigation.'
- The Multimodal Attack Surface: Unlike text LLMs, VLMs introduce an unprotected visual channel where attackers bypass text guardrails via visual features ([Nie et al., 2024](#)).

LLM Elasticity

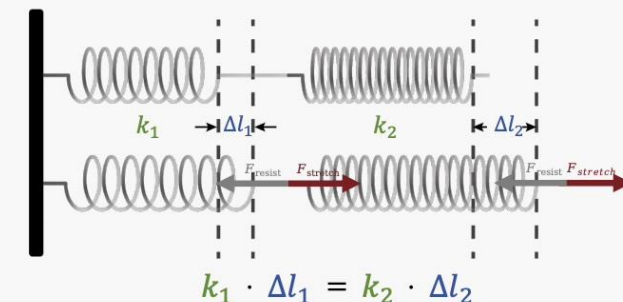
$$F_{\text{elastic}} \propto |\mathcal{D}_i| \cdot \Delta D_{\text{KL}}(p_{\theta} \| p_{\mathcal{D}_i})$$

([Ji et al., 2024](#)) shows that LLMs exhibit elasticity: a tendency to revert toward pretrained distributions during fine-tuning because the massive pretraining corpora exerts stronger influence than smaller alignment datasets.

- F_{elastic} is the strength of the dataset's tendency to pull the model toward its own behavior.
- $|\mathcal{D}_i|$ is the size of dataset $\mathcal{D}_i$
- p_{θ} is the model's current output distribution.
- $p_{\mathcal{D}_i}$ is the behavioral or token distribution represented by dataset $\mathcal{D}_i$
- ΔD_{KL} measures how different the model's current behavior is from that dataset's distribution.
- Since $|\mathcal{D}_p| \gg |\mathcal{D}_a|$ (pre-trained vs aligned data) => **Elastic reversion**

Our Contribution: The *Elasticity* of LLMs

Language models, fine-tuned with perturbations, exhibit an inverse relationship between **normalized compression rate changes** and **dataset volume**, akin to that of a series spring system, revealing the *elasticity* of LLMs.



Physical Model: The Hooke's Law

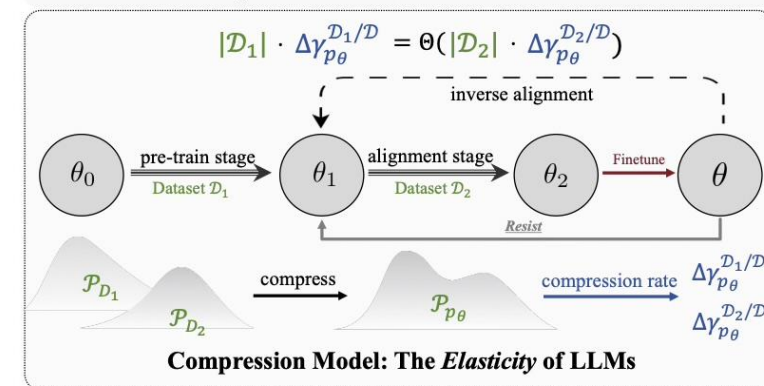


Figure 1: **The *Elasticity* of Language Models.** The change in normalized compression rates ($\Delta \gamma_{p_{\theta}}^{\mathcal{D}_i/\mathcal{D}}$) and the dataset volume ($|\mathcal{D}_i|$) follows an inverse proportionality law after perturbations, which is akin to the relationship between spring deformation (Δl_i) and stiffness (k_i) in coupled springs. We conjecture that the *elasticity* causes language models to resist alignment, enabling the possibility of *inverse alignment*.

VISAGE: Volumetric Index for Safety Alignment Guided by Explanation

A critical empirical finding is that safety basins have sharp boundaries: safety exhibits step-function collapse when crossing the boundary, with minimal graceful degradation. This geometry makes large parameter updates particularly dangerous.

- **The Safety Basin:** the average depth of a local region in the model's parameter space where the model remains safe.
- $B = \{\theta: S(\theta) \leq S_{threshold}, \theta \text{ connected to } \theta_{align}\}$
- A higher VISAGE score
 - model is safe in its current state
 - is also highly robust
 - less likely to have its safety alignment broken by fine-tuning or adversarial attacks

A. Safety basin universally appears in open-source LLMs' parameter spaces. Randomly perturbing model weights maintains safety level of original aligned model (light purple dot) in its local neighborhood.

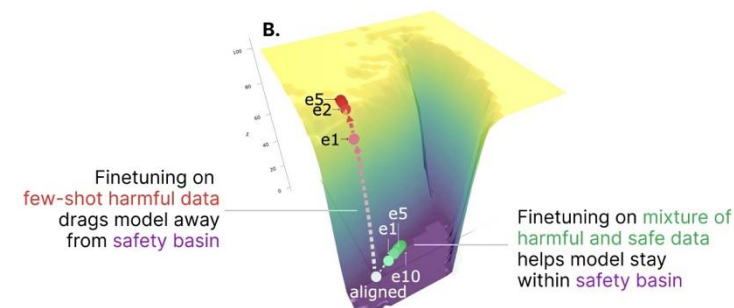
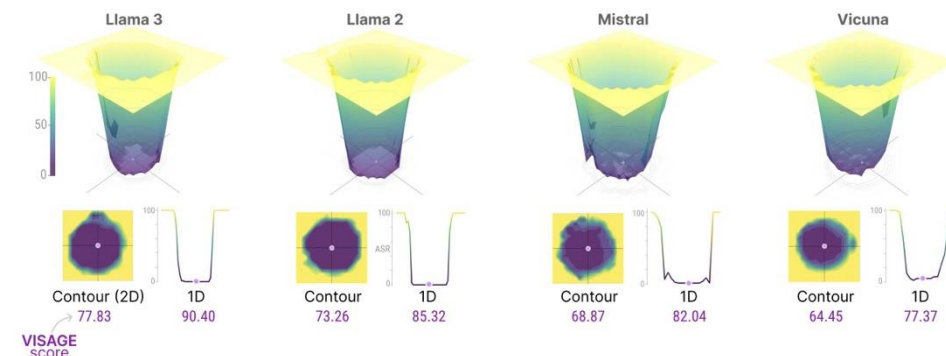


Figure 1: **A.** “Safety basin”, a new phenomenon observed universally in the model parameter space of popular open-source LLMs. Our discovery inspires us to propose the new VISAGE safety metric that measures the safety in LLM finetuning by probing its safety landscape. **B.** Visualizing the safety landscape of the aligned model also enables us to understand why finetuning with harmful data compromises safety but finetuning with both harmful and safe data preserves the safety.

$$\text{VISAGE} = \mathbb{E}_{\alpha \sim U(-a, a)} [S_{\max} - S(\alpha)] \quad \text{s.t. } S < S_{\max}$$

- *Safety / attack success rate (AdvBench, HarmBench)*
- *Truthfulness /Hallucination (TruthfulQA)*
- *Downstream task sequence (Dolly, GSM8K, MedMCQA, SQuAD v2)*

VLM Equivalent	What it is	LLM Equivalent
MM-SafetyBench	13 safety-critical scenarios with 5,040 text-image pairs.	AdvBench
FigStep	Embeds harmful instructions as image text alongside a benign text prompt (preserves diverse attack vectors)	No direct text-only equivalent. Attacks via <u>visual OCR channel</u> , completely bypasses text safety filters. Distinct attack shape.
JailBreakV-28K	Large-scale benchmark with 16 harm scenarios across 28k samples– mixing direct, contextual, and optimization-based attacks.	HarmBench
HarmBench (Text Slice)	Text-only attack queries to evaluate vulnerabilities in the VLM's language decoder. 510+ queries, Llama-Guard-3-8B judge	HarmBench
MMHal-Bench	Primary open-ended, model-graded hallucination benchmark. 8 question types per image across 12 images types	TruthfulQA
POPE	Secondary check using simpler yes/no object-hallucination probing.	TruthfulQA (Supplementary signal)
LLaVA-Instruct-150K	Benign general visual instruction-following to reduce excessive refusal before task-specific tuning.	Dolly
MathVista	Visual mathematical reasoning dataset. 6141 samples from 28 multimodal datasets	GSM8K
VQA-RAD and SLAKE	Medical visual question answering dataset. ~315 images, 3.5k Q/A pairs AND ~642 images, 14k Q/A pairs	MedMCQA
DocVQA	50k Document image reading comprehension and QA, 12k+ document images	SQuAD v2

VLM Attack Surface & Multi-Channel Evaluation

Attack surface	Example
Text channel	HarmBench or AdvBench prompts
Image-text rendering	FigStep-style image instructions
Visual semantic content	MM-SafetyBench
Optimization or jailbreak transfer	JailBreakV-style attacks
Cross-modal conflict	benign text + harmful image, or vice versa
OCR pathway	harmful text rendered in different fonts, layouts, or rotations

You should not rely on one aggregate ASR.

Report:

$$ASR_{text}, ASR_{image}, ASR_{cross}, ASR_{total}$$

This can reveal cases where text safety remains stable while visual safety collapses.

SOTA Baseline

Reference: Nie et al., *Safety Alignment for Vision Language Models*, [arXiv:2405.13581](https://arxiv.org/abs/2405.13581) (SafeVLM)

What SafeVLM Does (The Architectural Approach):

- Adds custom modules:
 - **Safety Projector**,
 - **Safety Tokens**, and
 - **Safety Head**.
- Requires a complex two-stage training process and alters model inference topology.

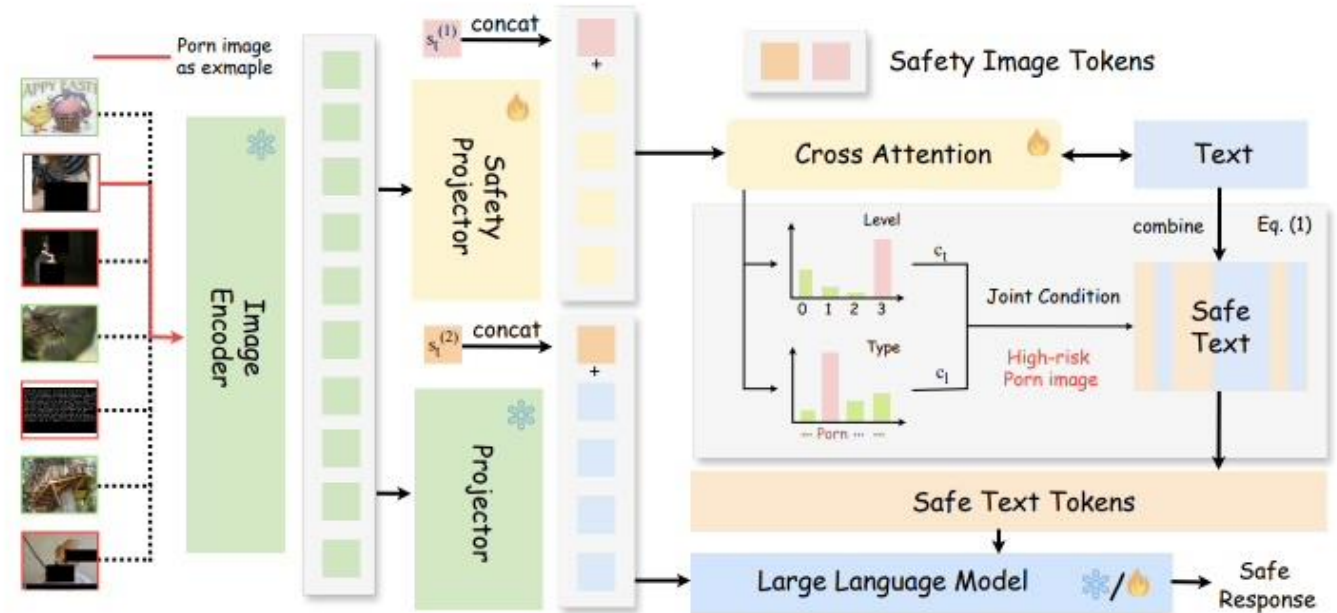


Figure 1: We conduct SafeVLM through two stages of training: (1) freezing the LLMs while learning safety features and adapting these features to align with LLM input, and (2) unfreezing the LLMs to enhance their understanding of unsafe content. During inference, we conditionally mix safety prompts with original inputs.

SafeVLM Empirical Results: Safety Scores & Over-Refusal Trade-offs (Tables 3 & 4).

Their Results on LLaVA-v1.5-7B

AdvBench (Vanilla ASR):

- Reduced from **6.45%** -> **1.72%**.

AdvBench (Suffix Injection):

- Reduced from **78.27%** -> **67.56%**.

XSTest Unsafe Rate:

- Dropped from **26.50%** -> **7.46%**
(shows significant over-refusal / false refusal risk).

RTVLM Safety Score:

- Increased from **6.27** -> **8.26**.

- Proves pre-trained projectors lack safety alignment, but architectural defense incurs high inference overhead and false refusals*

Table 3: **Comparison of text attack ASR (%) between the baseline model and the SafeVLM.** The ↓ denotes the ASR is lower the better and the ↑ denotes the ASR is higher the better.

Method	AdvBench		XSTest	
	Vanilla Instruction (↓)	Suffix Injection(↓)	Safe Instruction(↑)	Unsafe Instruction(↓)
LLaVA-v1.5-7B	6.45	78.27	91.20	26.50
SafeVLM	1.72 (↑4.73)	67.56(↑10.71)	76.89(↓14.31)	7.46(↑19.04)
SafeVLM (+LoRA)	1.90 (↑4.55)	69.86(↑8.41)	78.09(↓13.11)	6.96 (↑19.54)

Table 4: **Evaluation on the multimodal benchmarks**, including MMBench (Liu et al., 2023c), SEEDBench(Li et al., 2023b), and MME (Fu et al., 2024).

7B Method	MMBench	SEEDBench	MME ^P	MME	RTVLM
LLaVA-v1.5 7B	64.3	61.6	1487.9	1773.6	6.27
LLaVA-v1.5 7B + RT SFT	66.8	-	-	-	6.88
SafeVLM	66.8	65.3	1479.5	1762.7	8.18
SafeVLM (+LoRA)	68.5	63.7	1458.8	1753.8	8.26

VLM extension: Methodology

- We formalize **Continual Safety Alignment for VLMs** across a sequence of T multimodal tasks $\{D_1, D_2, \dots, D_T\}$
- Where $D_t = \{(x_i^v, x_i^t, y_i)\}_{i=1}^{N_t}$ contains visual inputs, text instructions, and target responses
- For a VLM, replace the language-model parameters with multimodal parameters:

$$\theta^{VLM} = (\theta_{vision_encoder}, \theta_{projector}, \theta_{LM_LoRA})$$

- If training only adaptors:

$$\theta^{train} = (\theta_{LoRA}, \theta_{projector})$$

With a vision encoder and base language model frozen

$$\Rightarrow \theta_t^{VLM} = \arg \min_{\theta} L_t^{VLM}(\theta; D_t^{image, text}) \text{ s.t. } \theta_t^{VLM} \in B_{VLM}$$

VLM extension: Methodology

- Text-only LLMs, G_i is calculated across LLM parameters or LoRA parameters.
- Modification In VLMs, you have three distinct module groups:
 - Vision Encoder (usually frozen during LoRA)
 - Vision-Language Projector / Connector (trainable cross-attention or MLP)
 - Language Model Backbone (trainable LoRA)
- If you compute gradients across everything, the vision encoder's high visual variance will completely swamp the gradient norm with image noise.

VLM extension: Methodology

Our 3 novel gradient attribution models

- **Our Adaptation:** We freeze the Vision Encoder and formulate **3 distinct gradient attribution modes:**

- $G_i^{(L)} = ||\nabla_{\theta_{LM_{LoRA}}} L_i ||_2$

(Language: Evaluates whether alignment drift is driven by conversational response priors in the language model.)

-

(Projector: Evaluates whether drift is driven by cross-modal feature warping justified by SafeVLM's finding that pre-trained projectors lack safety alignment)

- $G_i^{(J)} = \sqrt{(G_i^{(L)})^2 + \lambda \cdot (G_i^{(P)})^2}$

(Joint: combines both VLM signals with a balancing scalar)

VLM extension: Methodology

Language LoRA, Projector, Joint Multimodal Tension, Length normalization

- $G_i^{(L)} = \sqrt{G_1^2 + G_2^2 + \dots + G_k^2}$
 - This is the L2 norm (magnitude). It gives one single number showing how hard the language decoder is being pushed.
- $G_i^{(P)} = \sqrt{G_1^2 + G_2^2 + \dots + G_m^2}$
 - Shows how hard the visual-to-language bridge is being pushed by this image.
- $G_i^{(J)} = \sqrt{G_i^{(L)^2} + \lambda \cdot G_i^{(P)^2}}$
 - combining both tensions into one overall VLM score.
- $Normalized\ G_i = \frac{G_i^{(J)}}{\sqrt{target\ token\ count}}$ **(gradient per token)**
 - Prevents long answers from getting high scores just because they have more words.
 - WHY? - Eg, Sample A = yes, Sample B = 300 words. Even if both samples are equally safe and happy, Sample B will naturally have a much larger gradient norm G_i simply because you are adding up 300 gradient vectors instead of 1

VLM extension: Methodology

- Also record the combined norm:
 - $G_I^{total} = ||\nabla_{\theta_{LM_{LoRA}}, \theta_{projector}} L_i ||_2$
- One total norm will not know which mechanism is responsible => a high gradient could be caused by:
 - a text-output format mismatch
 - an image that is difficult to interpret
 - a poorly grounded visual representation
 - an unusually long target
- Examples groups for first set:
 - Low-G: lowest 20%
 - Moderate-G: samples closest to the median
 - High-G: highest 20%
 - Random: random 20%
 - All: all available samples
- The method is not simply take the lowest gradients. It first *removes extreme-loss* examples, *computes gradient norms* on the remaining candidates, and *selects samples nearest the median gradient norm*.

VLM extension: Methodology

Our 3 novel gradient attribution models

- In VLMs, the input contains visual tokens that must be strict label-masked to **label = -100** in PyTorch CrossEntropyLoss (visual patch tokens masked from loss) on **every single image patch token** and on the prompt text.
- Prevents the gradient from being dominated by image reconstruction (visual features) loss instead of task-response alignment.
- **The Result:** Gradients are computed strictly on the target answer tokens, meaning G_i measures true instruction alignment rather than image pixel noise.

Execution Plan

Stage	LLM Task	Our VLM Equivalent	Reasoning	Citation
1	Dolly (15K instruction following)	LLaVA-Instruct-150K	General visual instruction following , same role as Dolly (reduces refusal before task-specific tuning). Benign, diverse, general purpose.	Liu et al., arXiv:2304.08485
2	GSM8K (math word problems)	MathVista (6,141 problems)	Visual-mathematical reasoning from charts, diagrams, geometry. Direct multimodal analog to GSM8K. 6,141 problems	Lu et al., arXiv:2310.02255
3	MedMCQA (medical MCQ)	VQA-RAD (kept for held-out evaluation) or SLAKE (preferred for training)	Clinical image understanding: radiology, pathology. VQA-RAD = radiology QA pairs. SLAKE = bilingual medical VQA	Lau et al., Nature Scientific Data 2018 ; Liu et al., arXiv:2102.09542
4	SQuAD v2 (reading comprehension)	DocVQA (~50,000 Q&A pairs)	Dense document OCR and layout reasoning. Directly tests extractive reading comprehension on visual documents, same capability as SQuAD v2. ~50,000	Mathew et al., arXiv:2007.00398

Implementation Plan

- **Colab T4:**

- *Model:* Qwen2-VL-2B-Instruct (4-bit QLoRA, frozen ViT).
- *Pilot Run:* 1 task (LLaVA-Instruct-150K subset, N=1,000).
- *Core Hypothesis Test:* Does High- G_i induce higher ASR than Moderate- G_i on FigStep and MM-SafetyBench?
- *Audit:* Log $G_i^{(L)}$ *vs.* $G_i^{(P)}$ distributions.

- **PSU Campus Cluster:**

- Run full 4-task continual pipeline across 3 random seeds.
- Evaluate full benchmark suite (MM-SafetyBench, FigStep, JailBreakV-28K, MMHal-Bench).
- Execute ablation over *language LoRA alone* $G_i^{(L)}$, *projector alone* $G_i^{(P)}$, *and joint attribution* $G_i^{(J)}$

Practical Action Plan for Local-First Setup

- **Model Selection:** Qwen2-VL-2B-Instruct is ideal for Apple Silicon (MPS backend or mlx). It shares the Qwen family architecture analyzed in the paper and keeps memory requirements low.
- **LoRA Strategy:** Target q_proj, v_proj, k_proj, o_proj, and gate_proj, up_proj, down_proj in the LLM backbone with $r=16$, $\alpha=32$.
- **Phase 2 Proof-of-Concept Target:**
 - Base Model: Qwen2-VL-2B-Instruct.
 - Dataset: Single task (e.g., 2,000 samples of LLaVA-Instruct-150K or MathVista).
 - Metric to record: ASR on a subset of MM-SafetyBench + Task Accuracy before vs. after fine-tuning using:
 - Standard Fine-Tuning (All samples)
 - Random Selection (20% samples)
 - Moderate- G_i Selection (Ours, 20% closest to median G_i)
 - *Success Criterion:* Confirming that Moderate- G_i preserves lower ASR than Standard/Random fine-tuning while keeping comparable task performance on the VLM.

Modifications for VLMs

Disentangling Modality Tension (Format Mismatch vs. Visual Cross-Attention)

- The original paper identified high- G_i samples in text LLMs as format mismatches (e.g., short classification targets vs. verbose aligned outputs).
- In VLMs, high gradients arise from two distinct sources:
 - Output format mismatch (same as LLMs).
 - Visual grounding conflict: Complex images with dense visual information generate high gradient norms because the vision connector projections collide with LLM text priors.
- *Improvement:* Track G_i split by token types if possible, or log sample image complexity (e.g., image resolution, object count) during your Phase 2 audit to see if Moderate- G_i filtering preferentially removes complex visual scenes or standard format mismatches.

Modifications for VLMs

Safety Judges for VLMs

- Potentially using Llama-Guard-3-8B on the VLM's generated **text responses** for measuring safety output.
- For visual jailbreak attacks (FigStep, MM-SafetyBench), the judge model evaluates the *final generated text output* against the intended *harmful query embedded in the prompt/image*, not just the raw text prompt.

- A text-only judge can evaluate the generated answer, but it may not understand:
 - what harmful instruction was embedded in the image
 - whether the model followed or ignored the visual instruction
 - whether the model misread the image;
 - whether the answer is harmful only in relation to the image.

- Use a two-stage evaluation:

Stage 1: Automatic judge

- response safety classification
- refusal/compliance label

Stage 2: Manual audit

- attack recognized
- harmful request followed
- visual content understood
- partial compliance
- hallucination

- Store the raw response for every attack, not only the judge score.
- Also report refusal quality. A model that refuses every visual question may have low ASR but poor usefulness. Pair safety with capability:

Safety utility tradeoff = (*Low ASR, High TaskScore*)

Threat Model

- The attacker may provide:
 - a harmful text prompt;
 - a harmful instruction embedded in an image;
 - a benign text prompt paired with a malicious image;
 - a benign image paired with a harmful text prompt;
 - an image containing typography, screenshots, diagrams, or rendered text;
 - a jailbreak-style multimodal prompt.
- The attacker does **not** modify model weights during evaluation unless you explicitly study adversarial fine-tuning.
- Then organize attacks by attack surface. Attack evaluation must isolate visual vs. textual channels to detect modality-specific alignment collapse.

Critical Metrics, Datasets, and Observability Checklist

Dimension	Metric / Signal	Execution in Paper
Safety Preservation	ASR (Attack Success Rate)	Tested on AdvBench & HarmBench via Llama-Guard-3-8B (evaluates refusal vs. harmful execution).
	VISAGE Volumetric Score	Perturbs parameters across N random directions d at varying scalar steps α
Continual Dynamics	Backward Transfer (BWT)	$BWT = \frac{1}{T-1} \sum_{i=1}^{T-1} (R_{T,i} - R_{i,i})$ Measures retention of past tasks
	Forgetting Measure (FM)	$FM_k = \max_{l \in \{1 \dots T-1\}} (R_{l,k} - R_{T,k})$ Quantifies peak-to-final accuracy drop
	Task Interference Matrix	Off-diagonal accuracy changes when learning Task B after Task A
Data Diagnostics	KL Divergence Shifts	Measure $D_{KL}(p_{\theta_t} p_{\theta_{pretrain}})$ vs. $D_{KL}(p_{\theta_t} p_{\theta_{aligned}})$
	Sample Audits	Log per-sample token lengths, losses, and gradients to inspect filtered items.

Baseline	What it tests
Standard fine-tuning	Unconstrained safety degradation
Random 20% selection	Benefit of using fewer samples
Low- G_i selection	Safety-first end of gradient spectrum
High- G_i selection	Whether high gradients are specifically harmful
Moderate- G_i selection	Proposed method
Gradient clipping	Whether magnitude control is sufficient
KL regularization	Whether behavioral anchoring prevents drift
EWC	Whether generic continual-learning regularization is enough
LoRA/adapter baseline	Whether parameter-efficient tuning alone preserves safety
Safety-preserving VLM method	Comparison with multimodal safety-specific SOTA

Summary and Open Research questions

- **What We Know (LLMs):** High-gradient samples are format mismatches that drive elastic reversion out of safety basins; median filtering preserves alignment without extra safety data.
- **What We Are Testing (VLMs):**
 - Does multimodal gradient selection preserve VLM safety across both text and visual jailbreaks?
 - Does tension concentrate in the language LoRA weights $[G_i^{(L)}]$ or the cross-modal projector $[G_i^{(P)}]$?
 - Can we eliminate catastrophic forgetting in continual VLM domain adaptation with zero architecture overhead?
 - Can data-centric median filtering achieve competitive safety retention against architectural approaches like SafeVLM without inference penalties?
 - Does high-gradient sample selection drive safety basin collapse in multimodal models as it does in text LLMs?

References

- Bach et al. (arXiv:2604.17215)
- Ji et al. (arXiv:2406.06144)
- Peng et al. (arXiv:2405.17374)
- MM-SafetyBench
- FigStep
- SaLORA
- LARF
- EWC
- VLGuard
- LLaVA
- MathVista
- vLLM Engine